

AI SQL Query Builder — Natural Language to SQL

Domain: AI + Database Tools | AI/LLM Based Proof of Concept (POC)

1. Problem Statement

Non-technical users (teachers, managers, admins) ko apna database query karne ke liye SQL syntax nahi aata. Har baar developer se query likhwana time-consuming hai, aur simple reports ke liye bhi technical dependency banti hai. Isse decision-making slow ho jati hai jab data turant chahiye ho.

2. Proposed AI Solution

- User plain English/Roman Urdu mein sawal type karega, jaise 'Show me all students who scored above 80 in Math'.
- LangChain chain (prompt + LLM) is question ko database schema ke context ke sath process karke SQL query generate karega.
- Generated query ko safety-check kiya jayega (sirf SELECT allow, DELETE/DROP/UPDATE block) phir PostgreSQL par execute hogi.
- Result table format mein Streamlit par display hoga, sath mein generated SQL bhi dikhaya jayega transparency ke liye.

3. Tools & Technologies

LangChain (LCEL chain), OpenAI/Groq (LLM), FastAPI (backend), PostgreSQL + SQLAlchemy (database), Pydantic (structured/validated output), Streamlit (demo UI)

4. Dataset / Data Source

Koi bhi sample relational database (e.g. students/courses, ya company sales data) — schema hi input hai, bara dataset zaroori nahi.

5. Feasibility for POC

High — LangChain ka prompt + LLM pattern already practice kiya hua hai (Day 16-19), aur database schema chhota hone ki wajah se POC jaldi ban sakta hai.

6. Expected Impact

Non-technical staff khud apna data explore kar sakenge, developer dependency kam hogi, aur reporting turant ho sakegi.

Technical Defense — Sir Q&A Preparation

7. Kya Use Karoon Gi (My Tech Stack) & Kyun

- LangChain LCEL chain (prompt | model | output_parser) — exact wahi pattern jo maine Day 16 mein seekha, structured SQL output ke liye.
- Pydantic structured output — LLM se raw text ke bajaye validated SQL string return karwaon gi, taake malformed query na bane.
- FastAPI + PostgreSQL + SQLAlchemy — query execute karne aur query history log karne ke liye, jaisa maine user-management project mein kiya tha.
- Prompt Engineering (schema injection) — database ka table/column structure prompt mein pehle se de doon gi taake LLM sahi table names use kare.

8. Kya Use NAHI Karoon Gi & Kyun (Rejected Alternatives)

- Direct query execution bina validation ke — reject kiya kyunke agar LLM galti se DELETE/DROP suggest kare to data loss ho sakta hai; isliye whitelist approach (sirf SELECT) use karoon gi.
- Fine-tuning apna khud ka model — bohat resource-intensive hai; existing LLM (GPT/Groq) ko achay prompt ke sath use karna zyada practical hai POC ke liye.
- Multiple databases support ek sath — scope ko ek single PostgreSQL DB tak limit rakha POC clarity ke liye.

9. Kaise Implement Karoon Gi (Step-by-Step)

- Step 1: Sample database schema banana (students, subjects tables) PostgreSQL mein.
- Step 2: LangChain prompt template banana jismein schema description aur user question dono jayen.
- Step 3: LCEL chain build karna: prompt | ChatOpenAI/Groq | StrOutputParser (ya Pydantic parser).
- Step 4: FastAPI endpoint jo question accept kare, chain call kare, SQL validate kare, phir execute kare.
- Step 5: Result + generated SQL Streamlit par display karna, query history PostgreSQL mein save karna.

10. Possible Issues / Challenges & Mitigation

- LLM kabhi galat/ambiguous SQL generate kar sakta hai — isliye safety validation layer aur query preview zaroori hai.
- Complex joins wale questions mein accuracy kam ho sakti hai — schema description jitni clear hogi, utni behtar query banegi.
- SQL injection jaisa risk agar raw user input directly query mein chala jaye — isliye parameterized queries aur whitelist approach use karoon gi.

11. Anticipated Questions from Sir & Answers

Q: Agar AI galat SQL bana de to?

A: System sirf SELECT queries allow karega aur execute karne se pehle query preview user ko dikhayega — koi destructive operation (DELETE/DROP/UPDATE) allow nahi hogi.

Q: Yeh kaam kaise karta hai without training a custom model?

A: Prompt Engineering se — database schema ko prompt mein context ke tor par de deti hoon, LLM already SQL syntax janta hai, bas usay apne specific tables ka pata chal jata hai.

Q: Real production mein security kaise handle hogi?

A: Read-only database user create karoon gi (sirf SELECT permission), rate-limiting, aur JWT authentication add karoon gi taake sirf authorized users hi query bana sakein.

Project Architecture & Folder Structure

12. Folder Structure (Sir ke pattern par based)

```
ai_sql_query_builder/
├── src/
│   ├── query_builder/
│   │   ├── __init__.py
│   │   ├── models.py          # QueryLog table (id, user_question, generated_sql, executed, created_at)
│   │   ├── schemas.py         # QueryRequest, QueryResponse
│   │   ├── router.py          # POST /query/ask
│   │   ├── services.py        # LLM chain -> SQL generation -> execution logic
│   │   ├── utils.py           # schema_extractor(), sql_safety_check()
│   │   ├── database.py
│   │   ├── settings.py
│   │   ├── main.py
│   │   └── chains/
│   │       └── sql_chain.py    # LangChain LCEL chain: prompt | model | parser
│   ├── logs/
│   │   └── app.log
│   ├── streamlit_app.py
│   ├── requirements.txt
│   ├── .env
│   ├── .gitignore
│   ├── notes.ipynb
│   └── README.md
```

13. Har File Mein Kya Likha Hoga

- **models.py** — SQLAlchemy table definition (e.g. query_builder table: id, columns, timestamps).
- **schemas.py** — Pydantic models jo request body aur response ka shape/validation define karte hain.
- **router.py** — FastAPI endpoints yahan define hote hain — sirf request receive karke services ko call karta hai.
- **services.py** — Business logic: LLM chain call karna, DB query build/execute karna, result process karna.
- **utils.py** — Helper/reusable functions (e.g. prompt formatting, schema extraction, validation).
- **database.py** — SQLAlchemy engine, SessionLocal, aur Base declaration.
- **settings.py** — .env se DATABASE_URL, API keys load karta hai (Pydantic BaseSettings).
- **main.py** — FastAPI app instance, sab routers include karta hai, uvicorn isi ko run karta hai.
- **chains/sql_chain.py** — LangChain LCEL chain define karta hai: prompt template + LLM model + output parser ek sath pipe (|) operator se joray jate hain.

14. Data Flow — Sab Kaise Connect Hota Hai

- User Streamlit app mein plain English question type karta hai → request FastAPI ke router.py tak jati hai.
- router.py services.py ko call karta hai, jo chains/sql_chain.py ki LCEL chain invoke karta hai (schema + question prompt mein pass karke).
- LLM SQL query generate karta hai → utils.py ka sql_safety_check() function verify karta hai ke sirf SELECT hai.
- Validated query database.py ke SessionLocal se PostgreSQL par execute hoti hai.
- Result + generated SQL query dono schemas.py ke response model mein format hoke Streamlit ko wapis jate hain.

- Har question aur generated SQL PostgreSQL ke query_logs table mein save hoti hai audit ke liye.

15. pgAdmin Mein Kya Data Show Hoga

- pgAdmin mein **query_logs** table dikhega jab main.py chalega.
- Columns: id, user_question, generated_sql, execution_status, result_row_count, created_at.
- Query Tool se 'SELECT * FROM query_logs;' run karne par har asked question aur uska generated SQL history mein dikhega — yeh demo mein bohat impressive point hai sir ko dikhane ke liye.
- Sath hi 'students'/'subjects' jaisi sample tables bhi honge jin par actual queries chalengi.

16. Streamlit Interface Mein Kya Dikhega

- Streamlit app browser mein khulega — ek text input box 'Ask your question in plain English'.
- User sawal type karega (e.g. 'How many students scored above 80?') aur 'Generate & Run' button dabayega.
- Result: generated SQL query code-block mein dikhegi (transparency ke liye), phir uske neeche actual result table format mein.
- Sidebar mein query history bhi show ho sakti hai (past questions list, PostgreSQL se fetch karke).

17. uvicorn Chalane Par Kaunsi APIs Show Hongi (/docs)

- uvicorn src.main:app --reload chalane ke baad http://127.0.0.1:8000/docs par Swagger UI khulega.
- '/query' prefix ke andar: POST /query/ask (question -> SQL + result), GET /query/history (past queries), GET /health.
- 'Try it out' se POST /query/ask ko test kiya ja sakta hai — {"question": "..."} body de kar 'Execute' karne par turant SQL aur result dikhega.
- /redoc par same APIs ka documentation view bhi milega.

18. Additional Anticipated Questions from Sir

Q: LCEL kya hai aur yahan kyun use kiya?

A: LangChain Expression Language — pipe (|) operator se prompt, model, aur parser ko chain kiya jata hai. Yeh code ko clean aur readable banata hai, jaisa maine Day 16 mein seekha (prompt | model | output_parser).

Q: Agar schema change ho jaye (naya column add ho) to system kaise adapt karega?

A: utils.py mein ek schema_extractor() function banaoon gi jo dynamically PostgreSQL information_schema se current table structure fetch karega, taake prompt hamesha up-to-date schema use kare.

Q: Yeh tool kis industry mein use ho sakta hai?

A: Kisi bhi business mein jahan non-technical staff ko data access chahiye — HR, sales reporting, school admin, inventory management wagara.